

BÁO CÁO LAB 4

BYPASS KIỂM SOÁT PHÍA CLIENT TRÊN FORM ĐĂNG NHẬP

Họ tên: UNG VẮNG ĐẠI LÂM MSSV: 2380601202 Lớp: 23DATA1

Hình thức: Cá nhân. Mã nguồn: github.com/UngVangDaiLam/ATw-CSDL_Lab (thư mục lab4/demo-app).

⚠ **Lưu ý đạo đức:** Bài thực hành chỉ thao tác trên ứng dụng demo do chính mình dựng, nhằm hiểu vì sao phải kiểm tra dữ liệu phía server. Không áp dụng lên hệ thống của người khác khi chưa được phép.

1. Môi trường thực hành

App demo **Node.js + Express** (server.js) phục vụ một form HTML. Kiểm soát phía client đặt ở script.js: **username không để trống** và **password ≥ 6 ký tự** (kèm thuộc tính HTML `required`, `minlength="6"`). Server có hai endpoint để so sánh:

- **POST /api/login-insecure** — KHÔNG kiểm tra lại, tin rằng client đã lọc (mô phỏng ứng dụng có lỗ hổng).
- **POST /api/login-secure** — kiểm tra lại đúng bộ quy tắc ở server (cách làm đúng).

Chạy: `cd lab4/demo-app` → `npm install` → `npm start` → mở `http://localhost:4000`.

2. Quy trình bypass kiểm soát client

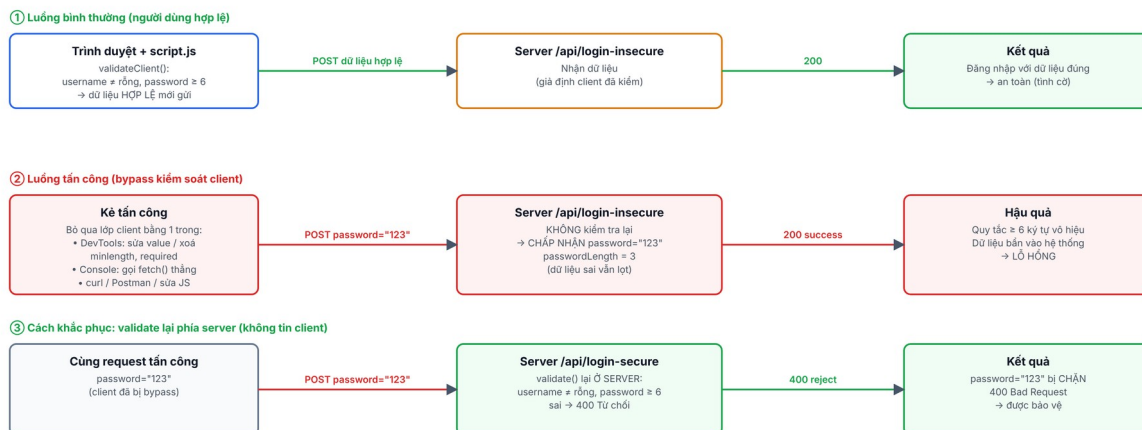
1. **Quan sát client chặn:** nhập username "admin", password "123" (3 ký tự). script.js hiện lỗi đỏ "Password phải có ít nhất 6 ký tự" và KHÔNG gửi request (Hình 2).
2. **Mở DevTools** (phím F12) → tab **Elements**: chọn ô password, xoá thuộc tính `minlength="6"` và `required`; xoá `required` của username. Đây là bước vô hiệu hoá ràng buộc HTML.
3. **Vô hiệu hoá JS validate:** trong tab **Console** gọi thẳng API, bỏ qua hàm `validateClient()`. Ví dụ:

```
fetch('/api/login-insecure', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ username: 'admin', password: '123' }) }) .then(r => r.json()).then(console.log)
```

4. **Quan sát server:** endpoint insecure trả `{ "success": true, ...passwordLength: 3 }` — dữ liệu sai quy định (3 ký tự) đã được CHẤP NHẬN. Việc bypass thành công (Hình 2).
5. **Đổi chứng:** gửi đúng dữ liệu "123" đó tới **/api/login-secure** → server trả **400** "Password phải có ít nhất 6 ký tự" → bị chặn. Cùng cách này có thể dùng curl/Postman mà không cần mở trình duyệt.

Bản chất: kiểm soát client chỉ chạy **trên máy người dùng** nên người dùng toàn quyền sửa/bỏ qua. Nó chỉ cải thiện trải nghiệm (báo lỗi nhanh), **không phải** là biện pháp bảo mật.

QUY TRÌNH BYPASS KIỂM SOÁT PHÍA CLIENT



Hình 1. Ba luồng: bình thường, tấn công bypass, và cách khắc phục bằng validate phía server

3. Rủi ro bảo mật khi không kiểm tra dữ liệu phía server

Rủi ro	Hệ quả
Dữ liệu sai định dạng lọt vào hệ thống	Mật khẩu quá yếu, email/sdt sai, dữ liệu bẩn làm hỏng logic và CSDL.
Vô hiệu mọi quy tắc nghiệp vụ	Vượt giới hạn số lượng, sửa giá tiền, đổi role/quyền, mua hàng giá 0đ...
Cổng cho các tấn công tiềm ẩn	Không lọc server → SQL Injection, XSS lưu trữ, command injection.
Lạm dụng tự động hoá	Kẻ tấn công gửi request hàng loạt bằng script (brute force, spam đăng ký).
Phá vỡ toàn vẹn và tin cậy	Hệ thống xử lý dữ liệu không hợp lệ, kết quả sai, khó kiểm toán.

Nguyên tắc nền tảng: **"Never trust the client"** — mọi dữ liệu từ client đều có thể bị giả mạo, nên server phải tự kiểm tra lại trước khi xử lý hoặc lưu.

4. Đề xuất giải pháp bảo vệ nâng cao

- **Validate lại ở server (bắt buộc):** kiểm tra kiểu, độ dài, định dạng, miền giá trị cho MỌI dữ liệu; dùng thư viện như Joi, express-validator, zod. Client validate chỉ để trải nghiệm.
- **Chuẩn hoá và lọc đầu vào:** chống injection bằng truy vấn tham số hoá (prepared statement), mã hoá/escape khi xuất ra HTML (chống XSS).
- **Xử lý xác thực đúng cách:** mật khẩu băm bcrypt, chính sách mật khẩu mạnh, giới hạn số lần thử (rate limit), khoá tạm, CAPTCHA.
- **Kiểm soát truy cập ở server:** xác thực và phân quyền cho từng API; không dựa vào việc ẩn/khoá nút trên giao diện.
- **Phòng thủ nhiều lớp:** HTTPS, security headers (helmet), CSRF token, ghi log và giám sát các request bất thường.

5. Kết luận

Thử nghiệm cho thấy kiểm soát phía client có thể bị vượt qua dễ dàng chỉ bằng DevTools hoặc một lệnh fetch, khiến endpoint không kiểm tra lại chấp nhận cả dữ liệu sai quy định. Vì vậy, kiểm soát client chỉ nên coi là tiện ích cho người dùng, còn ranh giới bảo mật thật sự phải nằm ở server: luôn validate lại, chống injection và áp dụng phòng thủ nhiều lớp.